1. It includes details of the exact Java exception that was thrown. Although not a vulnerability by itself, these kinds of details in outputs help a potential attacker to learn what technologies are being used to power an API. The headers are also leaking the version of the Jetty webserver that is being used by Spark under the hood. With these details the attacker can try and find known vulnerabilities to exploit. Of course, if there are vulnerabilities then they may find them anyway, but you've made their job a lot easier by giving away these details. Default error pages often leak not just class names, but full stack traces and other debugging information.
2. We are echoing back the erroneous input that the user supplied in the response, and not doing a good job of escaping it. When the API client might be a web browser, this can result in a vulnerability known as *reflected cross-site scripting* (XSS). See the sidebar below for more details on XSS attacks and how they related to APIs.
3. The Content-Type header in the response is set to `text/html` rather than the expected `application/json`. Combined with the previous issue, this increases the chance that an XSS attack could be pulled off against a web browser client.

We can fix the information leaks in point 1 by simply removing these fields from the response. In Spark, it's unfortunately rather difficult to remove the Server header completely, but you can set it to an empty string in a filter to remove the information leak: